

Software Requirements Specification (SRS)

Team : [Stack Attack](#)

Project: [Git Repository Health Dashboard using Cloud Environment](#)

Domain: [Cloud Computing & DevOps](#)

1. Problem Statement

Engineering teams lack a unified, real-time view of the health of their GitHub repositories. Without deep observability into their development lifecycle, they cannot proactively detect stagnating codebases, over-reliance on a single developer, PR review bottlenecks, or volatile code changes.

Currently, there is no lightweight, cloud-integrated tool that aggregates these signals from GitHub, computes a unified health score using a transparent rules-based engine, and persists historical trend data for long-term analysis — all without relying on hallucination-prone AI models.

2. Proposed Solution

We built a Cloud-Native Git Repository Health Dashboard. Our application connects to the GitHub REST API to fetch live repository telemetry (commits, pull requests, contributor data), processes it through a Deterministic Expert Rules Engine, persists health snapshots to AWS RDS (PostgreSQL), and visualizes historical trends via an embedded Grafana Cloud dashboard.

This provides engineering managers with clear, reproducible, and mathematically-grounded health signals for any GitHub repository — all within a single, unified Next.js application.

3. The New Innovation

- **The Deterministic Expert System:** We replaced hallucination-prone AI with a hard-coded "Expert Rules Engine." This engine applies strict, mathematical thresholds to the GitHub data. It generates verifiable DevOps recommendations (e.g., "High code churn (7,200 LOC changed). Consider breaking PRs into smaller, modular iterations.") that are 100% reproducible and auditable.

Four Core Health Metrics: We calculate four key DevOps health signals from live GitHub API data:

- **The Code Churn:** Measures total lines of code changed across the last 5 commits. When churn exceeds 5,000 LOC, it signals unstable, over-large changes that are too big to properly review.
- **The Bus Factor Risk:** Identifies if a project is overly dependent on a single developer (e.g., if one person makes more than 50% of commits, the project faces critical risk if they leave).
- **The Stagnation Risk:** Calculates the number of days since the last commit to detect repositories that are abandoned, blocked, or losing momentum.
- **The PR Bottlenecks:** Counts the number of open pull requests. When more than 5 PRs are waiting for review, the team is blocked and code review has become a bottleneck.
- **Key Engineer Profiler:** The dashboard identifies the top contributor by commit count and displays their GitHub avatar, username, and commit count — giving managers immediate visibility into single-point-of-failure risk.

4. Technology Stack

- **Frontend UI Framework:** Next.js 16 (React 19)
- **Styling Engine:** Tailwind CSS v4 (For premium, dark-mode glassmorphism design)
- **Backend Logic Engine:** Next.js API Routes (Node.js/Typescript)
- **Data Source:** GitHub REST API v3 (Live repository telemetry)
- **Database Management System:** AWS RDS PostgreSQL (via pg / node-postgres driver)
- **DevOps Observability:** Grafana Cloud (Time-series data visualization via SQL)
- **Query Language:** SQL (Used in Grafana to query PostgreSQL for trend data)

5. Cloud Integration: How We Use AWS

AWS RDS (Relational Database Service):

Usage: We provisioned a managed AWS RDS instance running PostgreSQL in the eu-north-1 region.

Purpose: This acts as the central source of truth for all repository telemetry. When our Rules Engine calculates a Health Score, it pushes a historical snapshot (repo_name, score, code_churn, stagnation_risk, bus_factor, generated_at) directly to AWS RDS. Grafana Cloud is securely connected to this database via SSL to generate live, time-series trend charts.

Grafana Cloud:

Usage: We provisioned a Grafana Cloud public dashboard connected to the AWS RDS PostgreSQL instance.

Purpose: Grafana renders historical trend charts showing how a repository's health score changes over time. The dashboard is embedded directly into the Next.js UI via an iframe, and is dynamically filtered using the selected repository name as a Grafana template variable (\$repo_name).

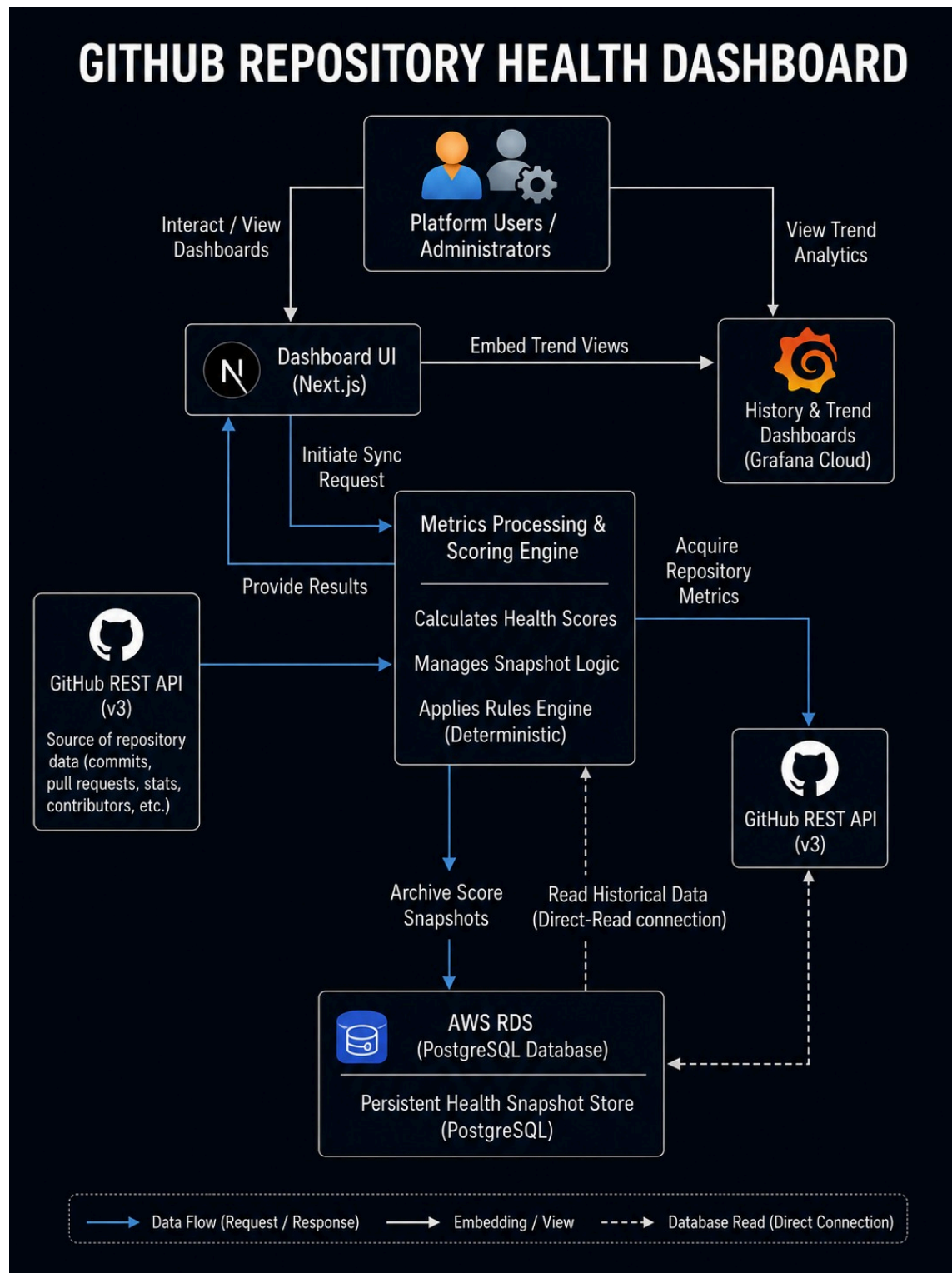
6. Application Workflow:

- **The Trigger:** The DevOps Manager selects a GitHub repository from the dropdown and clicks "Sync Telemetry" on the Next.js UI.
- **The API Fetch:** The Next.js backend calls the GitHub REST API to fetch the last 100 commits (author names, emails, timestamps), all open pull requests, and per-commit stats (lines added/deleted) for the last 5 commits.
- **The Rules Engine:** The fetched metrics are fed into the Deterministic Expert Rules Engine to calculate four health signals (Code Churn, Stagnation Risk, Bus Factor, PR Bottlenecks), compute the overall Health Score (0-100), and generate color-coded alerts and recommendations.
- **The Storage:** The final score and all metrics are pushed to the AWS RDS PostgreSQL database as a timestamped health snapshot.
- **The Visualization:** The Next.js UI updates instantly with the new Health Score gauge, four metric cards, the Key Engineer Profiler, and the Rules Engine alerts panel. Grafana reads the new row in PostgreSQL to update the historical trend charts.

7. Expected Outcomes

A fully functioning, cloud-integrated DevOps observability dashboard that tracks metrics that matter to engineering managers (Bus Factor, Code Churn, Stagnation, and PR Bottlenecks). Demonstration of GitHub REST API integration, server-side data processing, and AWS cloud infrastructure (RDS + Grafana). Proof that a transparent, reproducible, deterministic rules engine can replace black-box AI for DevOps recommendations. A historical trend visualization layer via Grafana that persists data across sessions, enabling long-term project health monitoring.

Solution Architecture:



Architecture Flow:

